

제 6 주:

재귀

성적처리



강 지 훈

jhkang@cnu.ac.kr

충남대학교 컴퓨터공학과



[제 6 주 실습]

재귀 (Recursion)



□ 실습 목표

■ 이론적 관점

- 재귀의 이해
- 재귀적 문제를 재귀 알고리즘으로 표현하는 방법

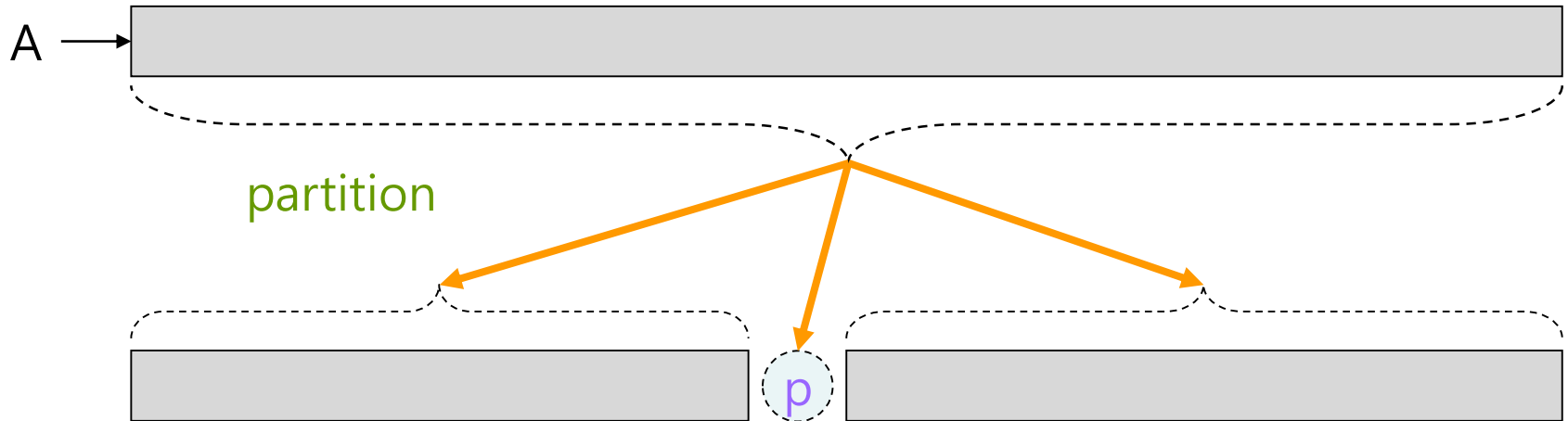
■ 구현적 관점

- 재귀를 Java 언어로 표현하는 방법
- 성적 처리 문제 해결에 사용되는 여러 가지 재귀적 기능을 이해하고 구현

퀵 정렬 (Quick Sort)



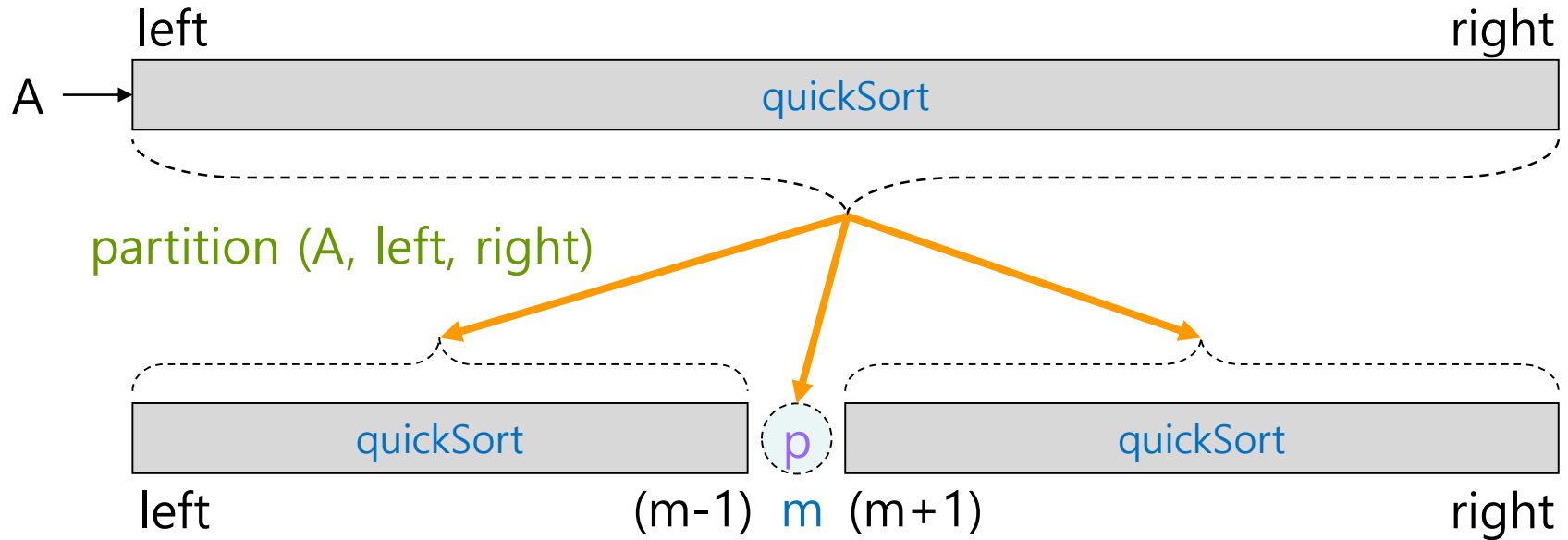
□ 퀵 정렬의 아이디어: 파티션



■ 파티션 (partition) 하기

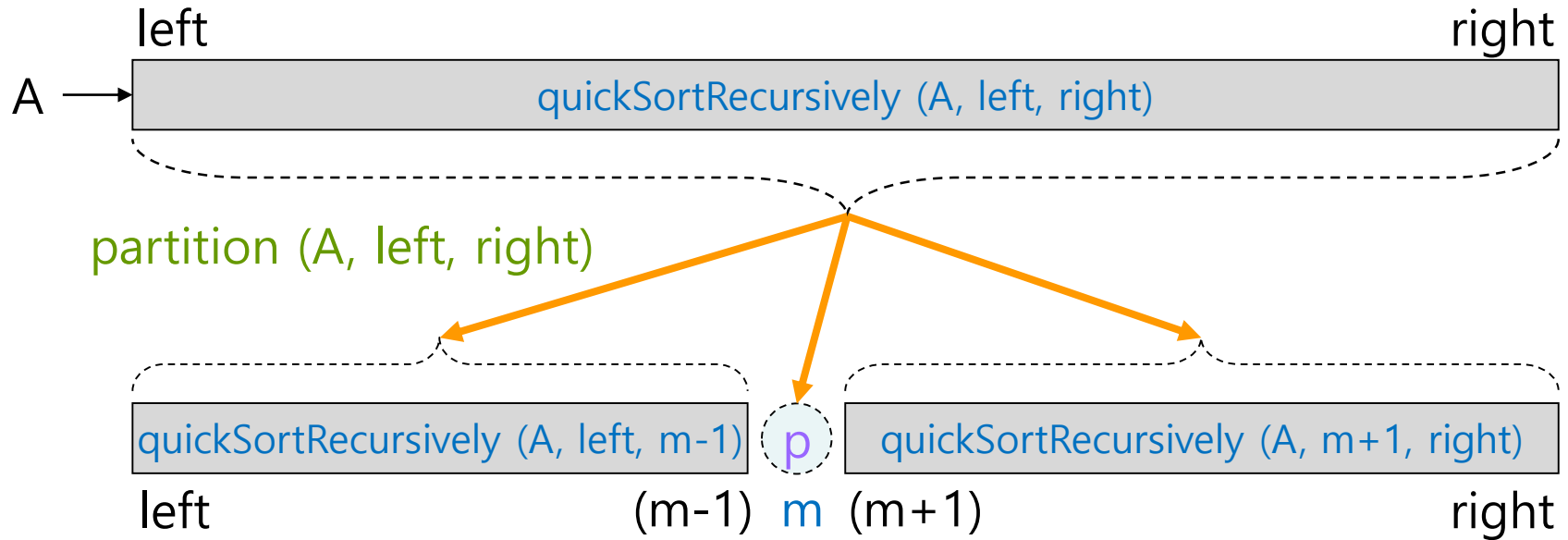
- 원소들을 특정 값을 기준으로 두 부분으로 나눈다.
 - ◆ 특정 값보다 작은 원소들은 왼쪽에, 큰 원소들은 오른쪽에 오게 하고, 그 사이에 특정 값이 놓이게 한다. (오름차순으로 정렬한다고 가정)
 - ◆ 특정 값을 **피벗 (pivot) 값**이라고 한다.

□ 퀵 정렬의 아이디어: 재귀적으로



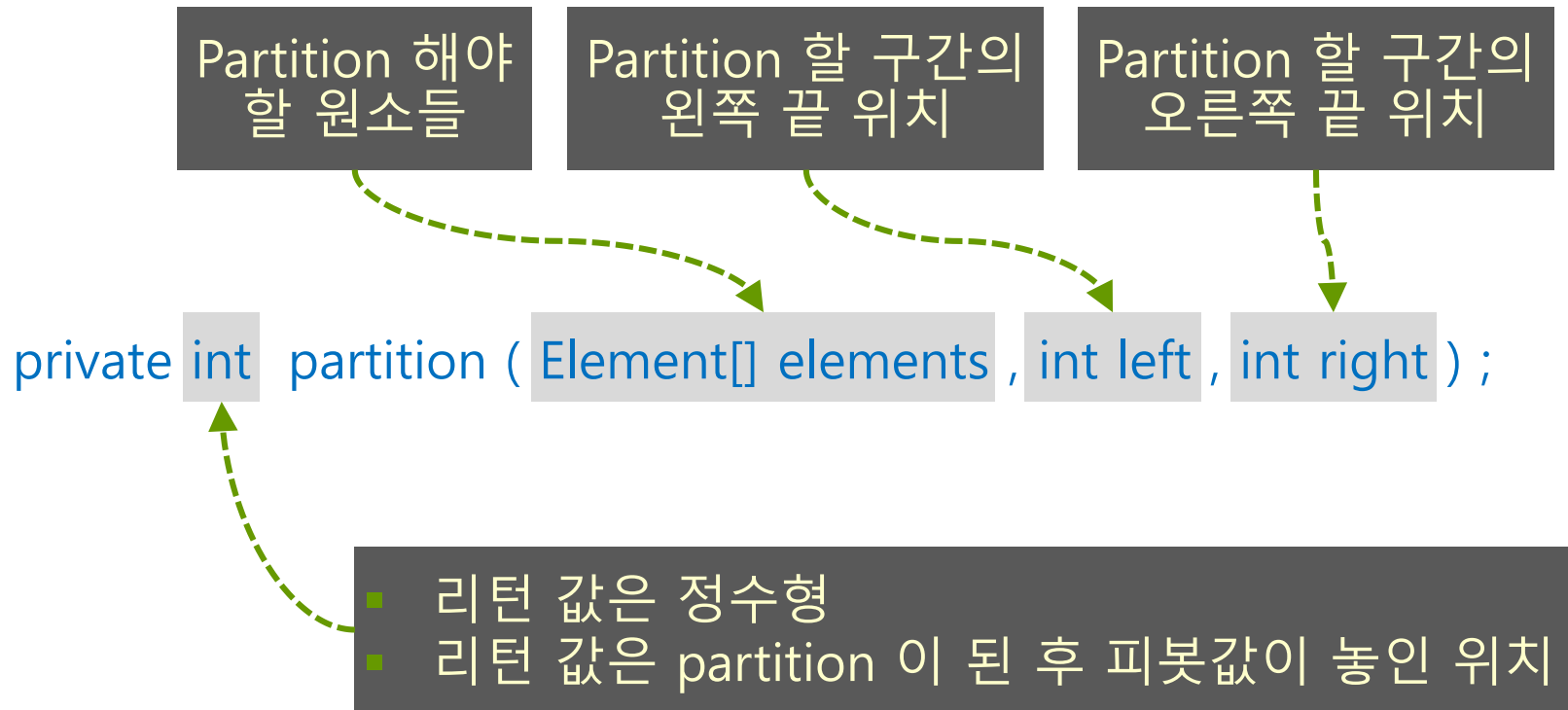
- 전체를 quickSort 하기 위해서는, partition 을 수행하여 만들어진 크기가 작아진 각각의 부분을 quickSort 한다.
 - 그러므로, 주어진 크기의 배열을 quickSort 하는 문제가, 크기가 작아진 두 개의 quickSort 문제로 바뀌었다.
- 탈출 (exit) 경우
 - quickSort 할 구간의 크기가 0 또는 1 이면, 그 자체가 이미 정렬이 되어 있는 상태이다.

□ 재귀 알고리즘 개요



```
public void quickSortRecursively (Elements[] A, int left, int right)
{
    if ( left < right ) { // 구간의 크기가 2 이상이면
        int mid = partition (A, left, right);           // DIVIDE
        quickSortRecursively (A, left, mid-1);          // CONQUER
        quickSortRecursively (A, mid+1, right);          // CONQUER
    }
}
```

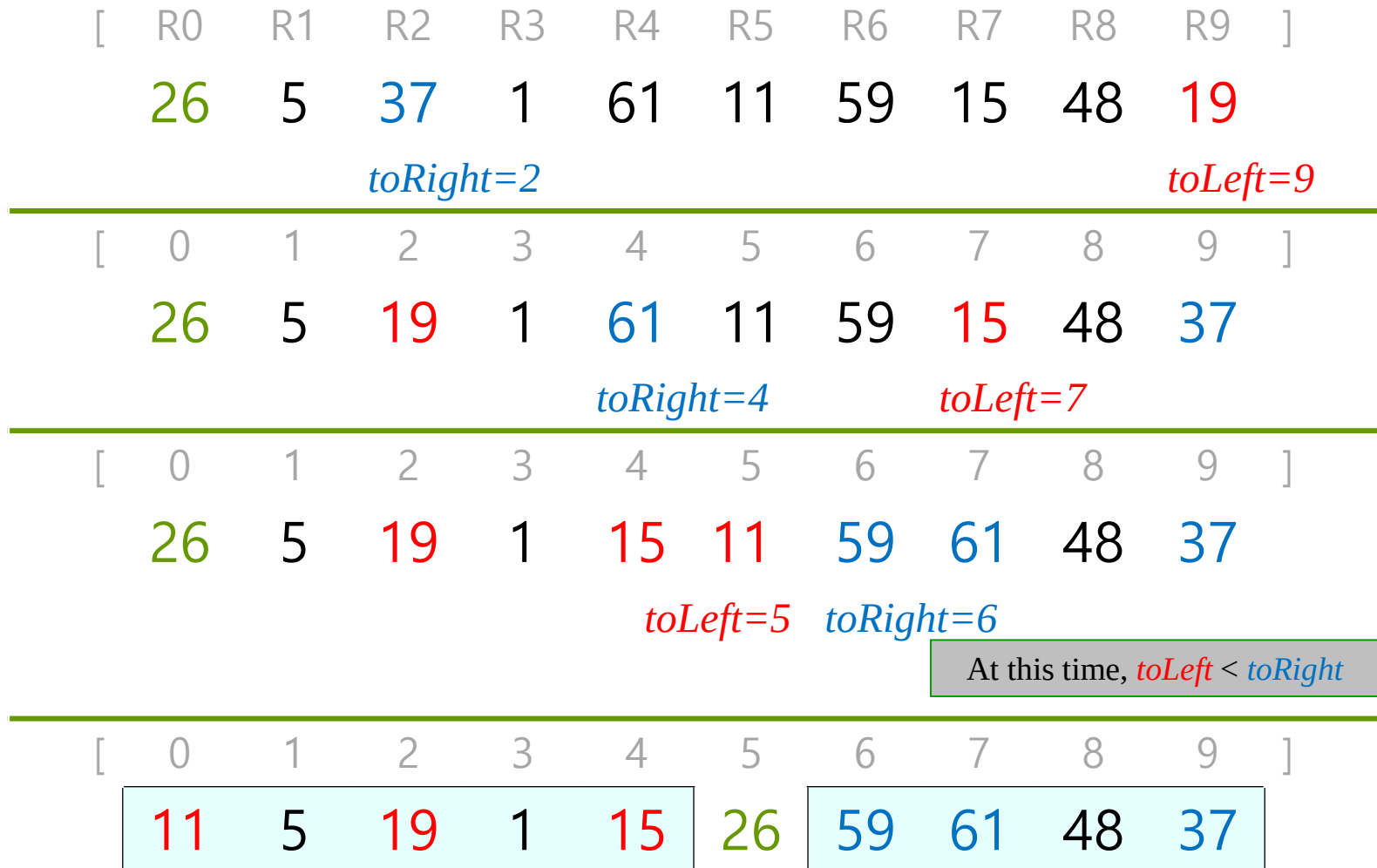
함수 partition()



□ 피벗 값 정하기

- 구간 안의 아무 원소를 피벗으로 정해도 된다
- 간편한 방법: 가장 왼쪽 값을 택하여 피벗 값으로 정한다.
 - $A[\text{left}]$ 가 피벗 값이 된다.

□ 파티션의 예



함수 partition()

```
private int partition (Element[] elements, int left, int right)
{
    int pivot = elements[left] ;
    int toRight = left ;
    int toLeft = right + 1 ;
    do {
        do { toRight ++ } while ( elements[toRight] < pivot ) ;
        do { toLeft -- } while ( elements[toLeft] > pivot ) ;
        if ( toRight < toLeft ) {
            swap (elements, toRight, toLeft) ;
        }
    } while (toRight < toLeft) ;
    swap (elements, left, toLeft) ;
    return toLeft ; // 피벗 위치가 toLeft 이다
}
```

□ 완성된 퀵 정렬 재귀 알고리즘

// Partition을 함수 형태로

```
public void quickSortRecursively (Element[] elements, int left, int right)
{
    if ( left < right ) {
        // 파티션
        // pivot 의 왼쪽에는 pivot 의 key 값보다 작은 원소들이 오게 하며,
        // pivot 의 오른쪽에는 pivot 의 key 값보다 큰 원소들이 오게 한다 ;
        int mid = partition(elements, left, right) ; // 파티션 후의 피벗 위치

        quickSortRecursively (elements, left, mid-1) ;
        quickSortRecursively (elements, mid+1, right) ;
    }
}
```

예: 퀵 정렬

R_0	R_1	R_2	R_3	R_4	R_5	R_6	R_7	R_8	R_9	R_{10}	l	r
(26	5	37	1	61	11	59	15	48	19)	$+\infty$	0	9
(11	5	19	1	15)	26	(59	61	48	37)		0	4
(1	5)	11	(19	15)	26	(59	61	48	37)		0	1
()1	(5)	11	(19	15)	26	(59	61	48	37)		0	-1
1	(5)	11	(19	15)	26	(59	61	48	37)		1	1
1	5	11	(19	15)	26	(59	61	48	37)		3	4
1	5	11	(15)	19()	26	(59	61	48	37)		3	3
1	5	11	15	19()	26	(59	61	48	37)		5	4
1	5	11	15	19	26	(59	61	48	37)		6	9
1	5	11	15	19	26	(48	37)	59	(61)		6	7
1	5	11	15	19	26	(37)	48()	59	(61)		6	6
1	5	11	15	19	26	37	48()	59	(61)		8	7
1	5	11	15	19	26	37	48	59	(61)		9	9
1	5	11	15	19	26	37	48	59	61			

□ 최종 퀵 정렬 알고리즘

```
private void partition (Element[] elements, int left, int right) {
    int pivot = left ; // pivot 원소의 위치를 정한다 ;
    int toRight = left ;
    int toLeft = right+1 ;
    do {
        do { toRight ++ ; } while ( elements[pivot] > elements[toRight] ) ;
        do { toLeft-- ; } while ( elements[pivot] < elements[toLeft] ) ;
        if ( toRight < toLeft ) {
            swap (elements, toRight, toLeft) ;
        }
    } while (toRight < toLeft) ;
    swap (elements, pivot, toLeft) ; // pivot 과 toLeft 위치의 원소를 맞바꾼다
    return toLeft ; // 파티션 후의 피벗 위치를 돌려준다 ;
}
```

```
private void quickSortRecursively (Element[] elements, int left, int right) {
    if ( left < right ) {
        int mid = partition (elements, left, right) ;
        quickSortRecursively (elements, left, mid-1) ;
        quickSortRecursively (elements, mid+1, right) ;
    }
}
```

```
public void quickSort (Element[] elements, int size) {
    if ( size > 1 ) {
        int maxLoc = 0 ;
        for ( int i = 1 ; i < size ; i++ ) {
            if ( elements[i] > elements[maxLoc] ) {
                maxLoc = i ;
            }
        }
        swap (elements, maxLoc, size-1) ;

        quickSortRecursively (elements, 0, size-2) ;
    }
}
```

```
private void swap(Element[] elements, int left, int right)
{
    Element temp = elements[left] ;
    elements[left] = elements[right] ;
    elements[right] =temp ;
}
```

과제에서 해결할 문제



□ 재귀적 문제 풀이

- 어떤 복잡한 문제들은 재귀 (recursion)을 사용하여 그 해결 방법을 아주 간결하게 표현할 수 있다.
 - 그러한 문제들은 그 자체가 재귀적(recursive)으로 정의된다.
 - 즉, 원래 문제보다 크기가 작은 같은 형태의 문제의 답을 안다고 할 때 원래 문제의 답을 알 수 있다면, 우리는 그러한 문제를 재귀적으로 풀 수 있다.

□ [프로그램 06]

■ 문제

- 한 과목 성적만 입력 받아 처리
- 성적 순으로 정렬한 다음, 출력한다.
- 평균, 평균 이상인 학생 수, 최고점, 최저점을 계산한다.
- 평균, 평균 이상인 학생 수, 최고점, 최저점을 출력한다.

□ 이 과제에서 필요한 객체는?

- ApplicationController
- AppView

- Model
 - Ban
 - Student
 - GradeCounter

입출력

입력:

- 매번 학번과 성적을 입력할지 (Yes/No) 여부를 먼저 묻는다.
- Yes 이면 한 학생의 학번과 점수를 입력 받는다.
 - ◆ 성적이 0보다 작거나 100 보다 크면 오류 메시지를 내보내고, 입력은 무시한다.
- No 이면 입력을 종료한다.
- 최대 학생 수 이상 입력되면, 공간 부족 메시지를 내보내고 종료한다.

출력

- 성적이 하나도 입력되지 않았으면 아무 일도 하지 않고 종료한다.
- 성적이 입력되었으면,
 - ◆ 입력된 학번과 점수와 그에 해당하는 학점을 출력한다.
 - ◆ 평균점과 평균 이상인 학생의 학생 정보를 출력한다.
 - ◆ 마지막에 학점 별 학생수를 출력한다.

출력의 예

<<성적 처리를 시작합니다.>>

성적을 입력하려면 'Y' 또는 'y'를, 종료하려면 다른 아무 키나 치시오 : y

-점수를입력하시오 : 82

성적을 입력하려면 'Y' 또는 'y'를, 종료하려면 다른 아무 키나 치시오 : y

-점수를입력하시오 : 45

성적을 입력하려면 'Y' 또는 'y'를, 종료하려면 다른 아무 키나 치시오 : y

-점수를입력하시오 : 93

성적을 입력하려면 'Y' 또는 'y'를, 종료하려면 다른 아무 키나 치시오 : y

-점수를입력하시오 : 102

ERROR : 0보다 작거나 100보다 커서, 정상적인 정수가 아닙니다.

성적을 입력하려면 'Y' 또는 'y'를, 종료하려면 다른 아무 키나 치시오 : y

-점수를입력하시오 : 66

성적을 입력하려면 'Y' 또는 'y'를, 종료하려면 다른 아무 키나 치시오 : y

-점수를입력하시오 : 87

성적을 입력하려면 'Y' 또는 'y'를, 종료하려면 다른 아무 키나 치시오 : y

-점수를입력하시오 : -1

ERROR : 0보다 작거나 100보다 커서, 정상적인 정수가 아닙니다.

성적을 입력하려면 'Y' 또는 'y'를, 종료하려면 다른 아무 키나 치시오 : N

[성적 입력을 종료합니다.]

[성적 입력을 종료합니다.]

평균 점수는 74.6 입니다.

평균 이상인 학생은 모두 3 명 입니다.

최고점은 93 점 입니다.

최저점은 45 점 입니다.

A 학점은 모두 1 명 입니다.

B 학점은 모두 2 명 입니다.

C 학점은 모두 0 명 입니다.

D 학점은 모두 1 명 입니다.

F 학점은 모두 1 명 입니다.

학생들의 성적순 목록입니다.

점수 : 93

점수 : 87

점수 : 82

점수 : 66

점수 : 45

<< 성적 처리를 종료합니다 >>

main()



□ main()을 위한 class

```
public class DS1_06_학번_이름 {  
    public static void main (String[] args)  
    {  
        ApplicationController appController = new ApplicationController() ;  
        // ApplicationController 가 실질적인 main class 이다.  
        appController.run() ;  
        // 여기 main()에서는 앱 실행이 시작되도록 해주는 일이 전부이다.  
    }  
}
```

Class "AppController"



□ Class "AppController" 의 구현 [1]

```
public class AppController {  
    // 비공개 변수들  
    private AppView _appView ;  
    private Ban _ban ;  
  
    // 생성자  
    public AppController()  
    {  
        this._appView = new AppView() ;  
    }  
  
    // 비공개함수의 구현  
    .....  
  
    // 공개함수의 구현  
    .....  
} // End of class "AppController"
```


□ Class "AppController" 의 구현 [2]

- 학생들의 정보를 입력 받아 Ban 객체에 저장한다 .

```
private boolean inputAndStoreStudents ()
{
    this.showMessage (MessageID.Notice_StartMenu) ;
    int score ;
    boolean storingAStudentWasSuccessful = true ;
    this._ban = new Ban() ;
    while ( storingAStudentWasSuccessful &&
           this._appView.inputDoesContinueToInputNextStudent() )
    {
        score = this._appView.inputScore() ;
        if ( score < 0 || score > 100 ) {
            this.showMessage (MessageID.Error_InvalidScore) ;
        }
        else {
            Student aStudent = new Student(score) ;
            this._ban.add (aStudent) ;
        }
    }
    this.showMessage (MessageID.Notice_EndMenu) ;
    return storingAStudentWasSuccessful ;
}
```

□ Class "AppController" 의 구현 [3]

- Ban 객체가 성적을 처리한 후, 그 결과를 얻어서 출력한다.

```
private void showStatistics ()
{
    this._appView.outputAverageScore (this._ban.averageScore()); ;
    this._appView.outputNumberOfStudentsAboveAverage
        (this._ban.numberOfStudentsAboveAverage()); ;
    this._appView.outputMaxScore (this._ban.maxScore()); ;
    this._appView.outputMinScore (this._ban.minScore()); ;

    // 학점 별 학생수는 Ban 객체로부터 GradeCounter 객체 형태로 얻는다
    GradeCounter gradeCounter = this._ban.countGrades();
    this._appView.outputGradeCountFor('A', gradeCounter.numberOfA()); ;
    this._appView.outputGradeCountFor('B', gradeCounter.numberOfB()); ;
    this._appView.outputGradeCountFor('C', gradeCounter.numberOfC()); ;
    this._appView.outputGradeCountFor('D', gradeCounter.numberOfD()); ;
    this._appView.outputGradeCountFor('F', gradeCounter.numberOfF()); ;
}
```

□ Class "AppController" 의 구현 [4]

- 성적 순으로 정렬된 학생 정보를 출력한다.

```
private void showStudentsSortedByScore ()  
{  
    this.showMessage (MessageID.Show_SortedStudentList) ;  
    for ( int position = 0 ; position < this._ban.size() ; position++ ) {  
        this._appView.outputStudentInfo  
            ( this._ban.elementAt(position).score() ) ;  
    }  
}
```

□ Class "AppController" 의 구현: run()

```

public class AppController {
    .....

    // 공개함수의 구현
    public void run()
    {
        this.showMessage (MessageID.Notice_StartProgram) ;

        // (1) 성적을 입력 받는다
        this.inputAndStoreStudents() ;

        if ( this._ban.isEmpty() ) {
            this.showMessage (MessageID.Error_NoInputScores) ;
        }
        else {
            // (2) 평균과 평균 이상인 학생수, 최고점, 최저점을 계산하고 출력한다.
            this.showStatistics();
            // (3) 성적순으로 정렬, 그리고 입력 받은 학생 정보를 출력한다.
            this._ban.sortStudentsByScore() ;
            this.showStudentsSortedByScore() ;
        }
        this.showMessage (MessageID.Notice_EndProgram) ;
    }
}

```

Class "AppView"



□ AppView: 인스턴스 변수, 생성자

```
public class AppView {  
    // 비공개 상수/변수들  
    private Scanner _scanner ;  
  
    // 생성자  
    public AppView ()  
    {  
        this._scanner = new Scanner(System.in) ;  
    }  
  
    // 공개함수의 구현  
    .....  
}
```

□ AppView: 입력을 위한 공개함수

■ 입력을 위한 공개 함수

- `public int inputInt()`
- `public String inputString()`
- `public Boolean inputDoesContinueToInputNextStudent()`
- `public int inputScore()`

□ Class "AppView" 의 공개함수

■ 출력을 위한 공개 함수

- public void outputAverageScore (float anAverageScore)
 - ◆ 평균값을 출력
- public void outputNumberOfStudentsAboveAverage (int aNumber)
 - ◆ 평균 이상인 학생 수 출력
- public void outputMaxScore (int aMaxScore)
 - ◆ 최고점 출력
- public void outputMinScore (int aMinScore)
 - ◆ 최저점 출력
- public void outputGradeCountFor (char aGrade, int aCount)
 - ◆ 각 학점에 대한 학생 수 출력
- public void outputStudentInfo (int aScore)
 - ◆ 학생들의 점수 출력
- public void outputMessage (String aMessageString)

□ Class "AppView" 의 구현 [1]

```
public class AppView {
    // 비공개 상수, 변수들
    ....
    // 생성자
    .....
```

이렇게 하는 이유는?

```
public int inputInt()
{
    return Integer.parseInt(this._scanner.nextLine());
}
```

```
public String inputString()
{
    return this._scanner.nextLine();
}
```

```
public void outputMessage (String aMessage) { ... }
```

Integer : Class 이므로,
객체 인스턴스는 null 이 가능
자체로는 산술 연산이 불가능
int : 직접 산술 연산이 가능

Integer.parseInt (String s) :
주어진 String 객체 s를 int 값으로 변환
(변환이 불가능하면 오류가 발생)

□ Class "AppView" 의 구현 [2]

```
public Boolean inputDoesContinueToInputNextStudent () {  
    char answer ;  
    System.out.print  
        ("성적을 입력하려면 'Y' 또는 'y' 를, 종료하려면 다른 아무 키나 치시오: ") ;  
    answer = this.inputString().charAt(0) ;  
    if ( (answer == 'Y') || (answer == 'y') ) {  
        return true ;  
    }  
    else {  
        return false ;  
    }  
}  
  
public int inputScore () {  
    int score ;  
    System.out.print ("- 점수를 입력하십시오: ") ;  
    score = this.inputInt() ;  
    return score ;  
}
```



□ Class "AppView" 의 구현 [3]

- 나머지 함수들은 각자 구현 할 것

Enum "MessageID"



Enum "MessageID"

```
public enum MessageID {  
  
    // Message IDs for Notices:  
    Notice_StartProgram,  
    Notice_EndProgram,  
    Notice_StartMenu,  
    Notice_EndMenu,  
  
    // Message IDs for Shows:  
    Show_SortedStudentList,  
  
    // Message IDs for Errors:  
    Error_WrongMenu,  
    Error_InvalidScore,  
    Error_NoInputScores  
  
} // End of enum "MessageID"
```

Class "Student"



Class Student [1]

```
public class Student {  
    private int _score ; // 점수
```

□ Student의 Member Functions

■ Student 의 Public Member function의 사용법

- public Student ()
- public Student (int givenScore)
 - ◆ 전달 받은 givenScore 로 학생의 점수를 초기화 하여 학생 객체를 생성한다.
- public int score ()
 - ◆ 학생의 점수를 전달한다.
- public void setScore (int newScore)
 - ◆ 학생의 점수를 newScore로 변경한다.

□ Student 의 Member Functions의 구현

■ Student 의 Public Member function의 구현

- public Student ()
 - ◆ 전달 받은 givenScore 로 _score 를 초기화
- public Student (int givenScore)
 - ◆ 전달 받은 givenScore 로 _score 를 초기화
- public int score()
 - ◆ _score 를 반환
- public void setScore (int newScore)
 - ◆ _score에 newScore 를 저장

Class "GradeCounter"



Class GradeCounter [1]

```
public class GradeCounter {  
    private int _numberOfA ;  
    private int _numberOfB ;  
    private int _numberOfC ;  
    private int _numberOfD ;  
    private int _numberOfF ;  
}
```

□ Member Functions

■ GradeCounter 의 Public Member function의 사용법

- `public GradeCounter()`
 - ◆ 학점을 셀 수 있도록 준비를 한다.
- `public void count (char aGrade)`
 - ◆ 주어진 학점을 받아서 해당 학점의 학생수를 증가시키게 한다.
- `public int numberOfA()`
 - ◆ 'A' 학점의 학생수를 얻는다.
- `public int numberOfB()`
 - ◆ 'B' 학점의 학생수를 얻는다.
- `public int numberOfC()`
 - ◆ 'C' 학점의 학생수를 얻는다.
- `public int numberOfD()`
 - ◆ 'D' 학점의 학생수를 얻는다.
- `public int numberOfF()`
 - ◆ 'F' 학점의 학생수를 얻는다.

□ Member Functions의 구현

■ GradeCounter 의 공개 함수의 구현

- public GradeCounter ()
 - ◆ 학점을 셀 수 있도록 준비를 한다.
- public int numberOfA ()

```
public int numberOfA() {  
    return this._numberOfA;  
}
```
- public int numberOfB ()
- public int numberOfC ()
- public int numberOfD ()
- public int numberOfF ()
 - ◆ "numberOfA()" 를 참고하여 나머지 함수들을 구현하시오.

□ Member Functions의 구현

■ GradeCounter 의 Public Member function의 구현

- 학점 세기, 학점 별 학생 수 얻기

```
public void count(char aGrade) {  
    switch (aGrade) {  
        case 'A':  
            this._numberOfA++;  
            break;  
        case 'B':  
            this._numberOfB++;  
            break;  
        case 'C':  
            this._numberOfC++;  
            break;  
        case 'D':  
            this._numberOfD++;  
            break;  
        default:  
            this._numberOfF++;  
    }  
}
```

Class "Ban"



□ Ban 객체의 구조 – 비공개 인스턴스 변수

```
public class Ban {  
    // 비공개 인스턴스 변수  
    private static final int DEFAULT_CAPACITY = 100 ;  
  
    private int _capacity ;  
    private int _size ;  
    private Student [] _elements ;
```



Ban 의 Member Functions

■ Ban 의 Public Member function의 사용법 [1]

- public Ban ()
- public Ban (int givenCapacity)
 - ◆ 생성자
- public int capacity()
 - ◆ 학급 객체가 가질 수 있는 최대 학생 수를 얻어 알아낸다.
- public int size()
 - ◆ 학급 객체가 가지고 있는 학생 수를 얻어 알아낸다.
- public boolean isEmpty()
 - ◆ 현재 학급이 비어있는지 확인한다.
 - ◆ 비어있으면 true 를 반환한다.
- public boolean isFull()
 - ◆ 현재 학급이 가득 차 있는지 확인한다.
 - ◆ 가득 차 있으면 true 를 반환한다.

□ Ban 의 Member Functions

■ Ban 의 Public Member function의 사용법 [2]

- `public boolean add (Student aStudent)`
 - ◆ 주어진 학생을 Ban 에 저장한다.
- `public Student elementAt (int anOrder)`
 - ◆ 주어진 순서의 학생 객체를 얻는다.
- `public void sortStudentsByScore ()`
 - ◆ 객체의 저장된 학생들의 성적을 성적 순으로 정렬시킨다.
- `public int minScore ()`
 - ◆ 학급의 최저점을 얻는다.
- `public int maxScore ()`
 - ◆ 학급의 최고점을 얻는다.
- `public float averageScore ()`
 - ◆ 학급의 평균 점수를 얻는다.

□ Ban 의 Member Functions

■ Ban 의 Public Member function의 사용법 [3]

- `public int numberOfStudentsAboveAverage()`
 - ◆ 평균 이상인 학생수를 센다.
- `public GradeCounter countGrades()`
 - ◆ 학점 별 학생수를 센다.

□ Ban 의 Member Functions

■ Ban 의 Private Member function

- private void swap (int positionA, int positionB)
- private void quickSortRecursively (int left, int right)
- private int partition (int left, int right)

- private float sumOfScoresRecursively (int left, int right)
- private int maxScoreRecursively (int left, int right)
- private int minScoreRecursively (int left, int right)

- private char scoreToGrade (int aScore)

□ Ban 의 Member Functions

■ Ban 의 Public Member function의 구현 [1]

- public Ban()
 - ◆ _capacity, _size, _elements를 초기화 한다.
- public Ban (int givenCapacity)
 - ◆ _capacity, _size, _elements를 초기화 한다.
- public int capacity()
 - ◆ this._capacity 를 반환한다.
- public int size()
 - ◆ this._size 를 반환한다.
- public boolean isEmpty()
 - ◆ this._size 가 0 인지 확인하여 반환한다.
- public boolean isFull()
 - ◆ this._size 가 this._capacity 보다 크거나 같은지 확인하여 반환한다.

□ Ban 의 Member Functions

■ Ban 의 Public Member function의 구현 [2]

- public boolean add (Student aStudent)
 - ◆ 성적을 배열에 저장
 - ◆ Bag 과 Set 을 구현한 것을 참고하여 구현
- public Student elementAt (int anOrder)
 - ◆ Ban 에는 원소 (즉, 성적)이 배열에 저장되어 있다.
 - ◆ 순서대로 나열된 원소들의 리스트로 생각하자.
 - ◆ 위치 anOrder 를 주면, 그에 해당하는 원소를 얻는다.

□ Ban 의 Member Functions

■ Ban 의 Public Member function의 구현 [3]

```
public void sortStudentsByScore() {  
    int size = this._size;  
    // 정렬할 데이터는 _this->_elements[0] 부터 _this->_elements[size-1] 까지 이다  
    // 퀵 정렬을 실행한다  
    if (size >= 2) { // 개수가 2 이상이면  
        // 최소값의 위치를 찾는다  
        int minLoc = 0;  
        for (int i = 1; i < size; i++) {  
            if (this._elements[i].score() < this._elements[minLoc].score()) {  
                minLoc = i;  
            }  
        }  
        // 최소값을 원소 구간의 맨 끝으로 옮긴다.  
        swap(minLoc, size - 1);  
        // 정렬을 시작한다  
        quickSortRecursively(0, size - 2);  
    }  
}
```

□ Ban 의 Member Functions

■ Ban 의 Public Member function의 구현 [4]

- public int minScore()
 - ◆ minScoreRecursively를 호출
- public int maxScore()
 - ◆ maxScoreRecursively를 호출
- public float averageScore()

```
public float averageScore() {  
    // 평균을 계산하여 return 값으로 돌려준다.  
    float sumofScores = (float) sumOfScoresRecursively(0, this._size - 1);  
    float average = sumofScores / (float) this._size;  
    return average;  
}
```


□ Ban 의 Member Functions

■ Ban 의 Public Member function의 구현 [5]

```
public int numberOfStudentsAboveAverage() {
    float average = averageScore();
    float score;
    int numbeOfStudentsAboveAverage = 0;
    for (int i = 0; i < this._size; i++) {
        score = (float) this._elements[i].score();
        if (score >= average) {
            numbeOfStudentsAboveAverage++;
        }
    }
    return numbeOfStudentsAboveAverage;
}

public GradeCounter countGrades() {
    // TODO Auto-generated method stub
    char currentGrade;
    GradeCounter gradeCounter = new GradeCounter();
    for (int i = 0; i < this._size; i++) {
        currentGrade = this.scoreToGrade(this._elements[i].score());
        gradeCounter.count(currentGrade);
    }
    return gradeCounter;
}
```

Ban: scoreToGrade()

```
private char scoreToGrade (int aScore)
{
    if ( aScore >= 90 ) {
        return 'A';
    }
    else if ( aScore >= 80 ) {
        return 'B';
    }
    else if ( aScore >= 70 ) {
        return 'C';
    }
    else if ( aScore >= 60 ) {
        return 'D';
    }
    else {
        return 'F';
    }
}
```



□ Ban: 퀵 정렬 [1]

```
private void swap(int positionA, int positionB) {
    Student temp = this._elements[positionA];
    this._elements[positionA] = this._elements[positionB];
    this._elements[positionB] = temp;
}
```

```
private int partition (int left, int right) {
{
    ..... // 이곳을 작성하시오
}
```

```
private void quickSortRecursively (int left, int right) {
    if ( left < right ) {
        int mid = this.partition (left, right) ;
        this.quickSortRecursively (left, mid-1) ;
        this.quickSortRecursively (mid+1, right) ;
    }
}
```



□ Ban: 합계, 최대값, 최소값

■ `private int sumOfScoresRecursively (int left, int right)`

// 성적의 합계를 재귀적으로 얻어서 돌려준다
// 크기를 (N-1)로 줄이는 재귀함수로 작성한다.

```
{
    if ( left > right ) {
        return 0; // 재귀의 탈출
    }
    else {
        return ( this._elements[left].score() +
                this.sumOfScoresRecursively(left+1,right) );
    }
}
```

■ `private int maxScoreRecursively (int left, int right)`

- 최고점을 찾아서 return 값으로 돌려준다.
- 두 개의 구간으로 나누는 재귀함수로 작성하시오.

■ `private int minScoreRecursively (int left, int right)`

- 최저점을 찾아서 return 값으로 돌려준다.
- 크기를 (N-1)로 줄이는 재귀함수로 작성하시오.

요약과 정리



□ 생각해 볼 점

- 학번 등 학생 정보를 추가하여 더 입력 할 것들이 있을 경우, 프로그램에서 바뀌어야 하는 부분은?
- 재귀적이지 않은 문제 풀이에 비해, 재귀적 문제 풀이가 항상 좋은 성능을 낸다고 할 수 있을까?
 - 최대값, 최소값 찾기
 - 퀵 정렬
 - 피보나치 수열 계산

□ 정리

- 실습 자료의 초록색으로 되어 있는 부분의 재귀적 구현 방법과 그렇게 프로그램을 작성한 이유를 작성하여 보고서에 포함 시키시오.

보고서 작성과 제출



□ PDF 보고서 작성 방법

■ 겉장

- 제목: 자료구조 실습 보고서
- [제06주] 재귀 - 성적처리
- 제출일
- 학번/이름

■ 내용

1. 프로그램 설명서

1. 주요 알고리즘 /자료구조 /기타
2. 함수 설명서
3. 종합 설명서

2. 프로그램 장단점 분석

3. 실행 결과 분석

1. 입력과 출력 (화면 capture 시킬 것)
2. 결과 분석

4. 소스코드

[제 6 주] 실습 끝



